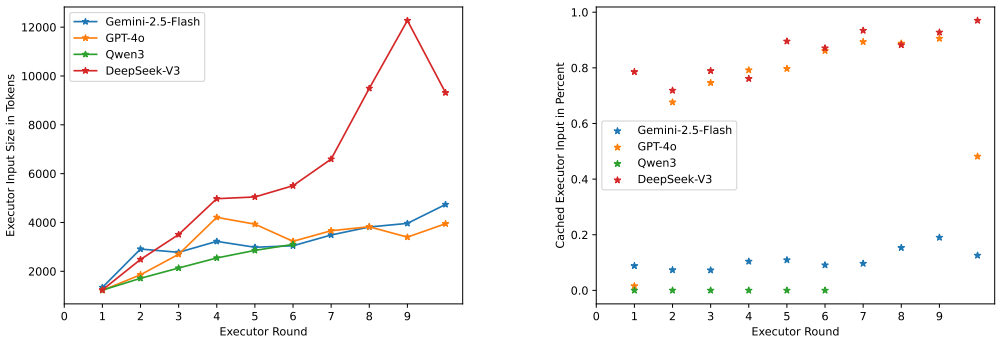

(a) Executor Input Prompt Size over Time.

(b) Percentage of Cached Input Prompts per Model.

Fig. 12. The Executor is given a task by the Planner and has up to 10 rounds to successfully finish this task. During each round, it can select new command line tool invocations to execute and analyzes the gathered result. Each round has all messages of previous rounds prefixed, thus the stored data grows over time. Modern LLM implement input prefix caching to reduce LLM operation costs.

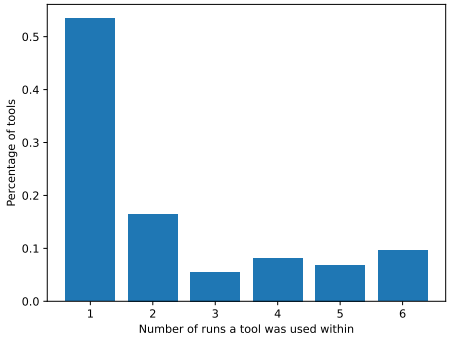

Fig. 13. Inclusion of Tools within OpenAI's o1+GPT-4o experiment runs. Please note, that this is an exact count. A tool that is accounted for with $n = 1$ is not included for $n = 2$. The graph indicates that many tools are only used within a single executor run while approximately 10% of tools were included in every sample.

The Executor often proposed invalid tool calls (35.9% on average). The first author, who is a professional penetration-tester, further separated those erogenous calls into two distinct classes: Type 1 errors are direct parameter errors. They happen if a mandatory parameter is not given, are typically directly detected by the tool, and instantly produce an error message. Type 2 errors occur when a parameter value is accepted by the tool even when it is semantically defective. We only count "obvious" errors as Type 2 errors, e.g., when *cat* or *ls* is used with a local non-existent directory, a random IP address is used as parameter, invalid hashes are passed to *hashcat* or *john* (e.g., "`<enter hash here.>`"), an invalid sql expression is used as subcommand for *impacket-mssqlclient* or an invalid RPC subcommand is used within *rpcclient*, or an invalid path is used within smbclient (\\server\share\dir instead of \\server\share). While Level 2 errors are another form of parameter errors, they are typically not detected by the tools themselves and are reported as network errors, and thus can "confuse" the Executor.

Table 8. Overview of Tool usage by OpenAI's o1+GPT-4o.

Command	% of runs	#	% errors	% Type 1	# Type 2	Command Description
Nxc and netexec	100%	244	46.72%	39.75%	6.96%	Multitool for SMB/LDAP,etc.
smbclient	100%	231	19.04%	6.49%	12.55%	Enumerating SMB shares, access files over SMB
cat	100%	100	21%	3%	18%	Outputting retrieved files
echo	100%	79	0%	0%	0%	Creating new files
nmap	100%	46	17.39%	10.86%	6.52%	Network scanner
rpcclient	66%	45	35.55%	4.44%	31.11%	Querying SMB resources
impacket-GetUserSPNs	100%	44	65.90%	13.63%	52.27%	Kerberoasting
john	100%	40	60%	5%	55%	Password Cracking
impacket-GetNPUsers	83%	37	48.64%	40.54%	8.10%	AS-REP Roasting
hashcat	83%	34	94.11%	0%	94.11%	Password Cracking
impacket-mssqlclient	33%	32	68.75%	43.75%	25%	Accessing Microsoft SQL Servers
impacket-smbexec	50%	23	69.56%	69.56%	0%	Executing Commands on remote servers over SMB
impacket-secretsdump	66%	21	9.52%	9.52%	0%	Dumping credentials from remote servers
impacket-getADUsers	66%	17	52.94%	52.94%	0%	Enumerating AD Users
ls	66%	17	0%	11.76%	11.76%	Listing Files

Command designated the executed command. *nxc* is an alias for *netexec* and thus both were grouped together. *% of runs* gives the percentage of runs within which a command was included while *#* gives the absolute count of command invocations. We detail the observed errors by given their total count (*% errors*) and further differentiate between syntactical errors (*%Type 1*) and semantical errors (*#2 Type 2*).

Of special note is the attempted usage of *hashcat* that failed in 94.11% of command invocations due to invalid hashes or an invalid hash format. *Impacket-mssqlclient* and *rpcclient* failed both due to invalid sub-commands given (68.75% and 35.55% respectively).

The full list of commands is provided in the appendix (Section D). Commands include typical penetration-testing tools on different abstraction levels that range from very specific ("low-level") tools such as *evil-winrm* or *certipy*, to broad ("high-level") tools such as *bloodhound-python*. Non-offensive tools such as compilers and interpreters (e.g., *python3*, *mono*/*mcs* and *pwsh*) were also employed during penetration-testing runs.

5.6.2 Mapping MITRE ATT&CK Tactics and Techniques. We mapped individual tasks within our samples to MITRE ATT&CK techniques and sub-techniques[24]. To increase clarity, we converted sub-techniques to their respective main techniques. Table 9 shows the ten most often used MITRE ATT&CK techniques and their respective tactics[25].

The tactics *Reconnaissance* and *Discovery* are typically used for network scans and domain enumeration. *Credential Access* is a broad tactic, including both password spraying attacks as well as gathering and abusing Kerberos Tickets or NTLM hashes. *Lateral Movement* designated directed attacks against network services.

[24] https://attack.mitre.org/techniques/enterprise/
[25] https://attack.mitre.org/tactics/enterprise/

Table 9. Mapping Tasks to MITRE ATT&CK Tactics and Techniques for OpenAI's o1+GPT-4o's runs.

MITRE Tactic	MITRE Technique	#	in %x runs	Examples
Credential Access	T1110: Brute Force	62	100%	Hashcast, nxc
Discovery	T1135: Network Share Discovery	43	100%	Nxc, smbclient
Credential Access	T1558: Steal of Forge Kerberos Tickets	26	100%	impacket-GetUserSPNs, impacket-GetNPUsers
Discovery	T1069: Permission Groups Discovery	19	83%	Ldapsearch, nxc, bloodhound
Discovery	T1615: Group Policy Discovery	17	83%	smbclient
Reconnaissance	T1595: Active Scanning	11	100%	nmap
Discovery	T1087: Account Discovery	9	66%	Ldapsearch, bloodhound, nxc
Credential Access	T1003: OS Credential Dumping	8	50%	Impacket-secretsdump, nxc
Lateral Movement	T1210: Exploitation of Remote Services	8	66%	Nxc, impacket-mssql
Credential Access	T1552: Unsecured Credentials	6	50%	smbclient

Professional Penetration-Testers categorized occurring commands into their respective MITRE ATT&ACK tactic and technique (which are sub-times of tactics). # gives the absolute amount of the respective techniques occurrences, *in % runs* gives the percentage of runs within the technique was detected and *Examples* highlights typical commands used for techniques.

Overall, the Top 10 techniques shown during our example runs describe an attacker that has gained an initial foothold into the Active Directory and now starts to perform lateral movement, execution, and privilege escalation. The different detected techniques and tactics indicate a healthy diversity of attack techniques and venues.

6 Discussion

This section discusses the quality of generated penetration testing plans/trajectories and commands, highlights opportunities for enhancements and future research, and finishes with a discussion on safety, ethics and defense.

6.1 The Problem with Qwen3

Quantitative analysis has shown that *Qwen3:32b* was not able to successfully compromise AD accounts (Section 5.2). The LLM was able to generate an initial PTT, select an appropriate next task, and successfully finish the given task—typically network and service enumeration— but was not able to integrate results back into the PTT. Typically, the updated PTT consisted of either a copy of the original PTT, an empty plan, or the Planner went off the rails and created a strategy for a new goal that diverted from the original penetration testing goal, e.g., "write incident response policies". As the PTT was not successfully updated, the Planner chooses the same task for the Executor over and over again, e.g., "perform a network scan for 192.168.56.0/24", leading to no overall progress. This behavior would not be redeemed by using techniques such as Retrieval-Augmented-Generation (RAG) as the problem is not insufficient background knowledge but lacking integration and summarization skills of the used LLM. The impact of this behavior can be seen in the PTT's growth or the lack thereof highlighted in Figure 11.

Another problematic behavior was that Qwen3 did not heed the safety instructions included in the scenario prompt (Section 3.2.5) and targeted both systems explicitly excluded such as the VM host machine as well as systems outside the test network.

Qwen3 also sometimes went "off the rails" by exchanging the penetration-testing goal with a new one: writing incident response policies. After devising a high-level incident plan, it deemed its task to be finished and stopped performing further steps. Qwen3 was the only evaluated LLM that routinely hallucinated facts such as successful exploitation of non-existing AD accounts with imagined passwords, or even successful compromise of the AD domain by reporting that "domain domination was achieved".

6.2 Planner: High-Level Attack Trajectories

Analysis indicates a substantial utilization of diverse attack tactics and techniques (Figures 9 and 13, Table 8). LLMs adhere to best practices by initiating operations with *Reconnaissance* and *Discovery* phases, subsequently exploiting vulnerabilities that align with the *Credential Access* and *Lateral Movement* tactics as defined by the MITRE ATT&CK framework. Tactics such as Execution and Privilege Escalation were observed less frequently, and in our scenario, they share considerable similarities with *Lateral Movement*. Despite variations in the individual attacks, the overall logical progression of the attack sequence remained consistent across multiple runs.

Models used diverse attack vectors for gaining initial access. After initial access was established, domain enumeration was typically conducted followed by credentialed attacks. Gathered Kerberos tickets and NTML hashes were cracked using *john* and *hashcat*. An overview of pursued attack vectors is shown in Figure 9.

Results indicate that all evaluated models have penetration-testing knowledge incorporated as part of their training corpus, negating the need to introduce specific penetration-testing knowledge through in-context learning or application of RAG. Involved penetration-testers stated that the pursued attack vectors are representative of vulnerabilities typically found in SME AD networks.

6.2.1 LLM comparison. Qwen3 was not able to update the PTT successfully (Section 6.1) and thus was not able to create good trajectories. o1 was able to compromise the highest amount of AD accounts of our our evaluated LLMs.

Comparing GPT-4o's log traces qualitatively with o1's log traces, the latter produced more concise PTTs as well as better task descriptions for the Executor. GPT-4o produced less efficient tasks, i.e., tasks using interactive, or performing network sniffing attacks that, while well-suited for the goal, consume a comparatively large amount of the allocated sampling time. In general, reasoning models (excluding Qwen3) performed 80% more high-level strategy rounds compared to non-reasoning models, indicating that they produced better task descriptions used by the Executor, allowing the latter to perform their tasks more efficiently (including the Auto-Fixing behavior highlighted in Section 6.4.3).

Qualitative analaysis of execution traces indicates that Gemini-2.5-Flash's trajectories were more stable than these produced by o1, i.e., it created similar sequences of tasks leading to similar trajectories containing the same compromised user accounts (or *almost-theres*). One side-effect of this was that Gemini-2.5-Flash always hyper-focused on the same DC (*MEREEN*) while o1 was switching between low-hanging fruits of multiple servers. All models except o1 were configured to use a temperature parameter of 0, i.e., to reduce variance between sampling runs. o1 does not support configuration of the used temperature. To verify the impact of temperature, we ran Gemini-2.5-Flash with a temperature of 0.8 which still kept its stable trajectories, indicating that while it is well-suited for specific tasks, a more creative and free-wheeling LLM might be advantageous for strategy-making.

6.2.2 Causal and Temporal Relationship between Tasks. PTTs included causal relationships between tasks. A typical example is a sequence of the Planner identifying various AD servers, performing attacks against those servers to gain an initial user password hash, turning it over to *hashcat* or *john* for password cracking, and finally yielding plain-text credentials that then can be used to perform additional authenticated attacks against the AD.

While heeding causal dependencies was a common occurrence, LLMs sometimes performed attacks too early, e.g., GPT-4o tried to perform Kerberoasting—an attack that depends on known AD credentials—without having compromised domain credentials before.